Blatt 5

Grundlagen der IT-Sicherheit

Luis Staudt

Aufgabe 1: AS mit asymmetrischen Schlüsselpaaren

Das Authentifizierungssystem wird als 5-Tupel $AS = (I, C, Z, F, L)$ mit zusätzlicher Subjektmenge S beschrieben. Anstelle eines gehashten Passwortes wird der öffentliche Schlüssel des Benutzers beim Server hinterlegt. Der private Schlüssel verbleibt ausschließlich beim Client.

Mengen und Funktionen

- **I**: Raum der Benutzernamen (z. B. zar).
- **S**: Menge der registrierten Benutzer, zu denen jeweils ein öffentlicher Schlüssel hinterlegt ist, z. B. $S = \{root, zar, \dots\}$.
- **Z**: Zuordnungsfunktion $Z : I \to S$. Prüft, ob der übergebene Benutzername registriert ist, und liefert das zugehörige Subjekt oder $\emptyset$ zurück.
- **C**: Menge der gültigen Signaturen über einer vom Server erzeugten Challenge (Zufallswert + Zeitstempel). Jede Signatur ist nur für genau eine Challenge gültig.
- **F**: Enthält die Signaturfunktion f des Clients. $f(s, challenge)$ fragt beim Benutzer den privaten Schlüssel (bzw. dessen Freigabe per PIN) ab und gibt die Signatur $sig = sign(sk_s, challenge)$ zurück.
- **L**: Enthält die Verifikationsfunktion $l : S \times C \to \{wahr, falsch\}$. $l(s, sig) := Z(s) \neq \emptyset \wedge verify(pk_s, challenge, sig)$.

Ablauf

Registrierung:

1. Der Benutzer erzeugt lokal ein Schlüsselpaar (sk, pk).
2. Der private Schlüssel sk bleibt beim Benutzer (z. B. geschützt durch eine PIN).
3. Der öffentliche Schlüssel pk wird zusammen mit dem Benutzernamen an den Server übertragen und gespeichert.

Anmeldung:

1. Client sendet seinen Benutzernamen i an den Server.
2. Server prüft $Z(i)$. Ist der Benutzer unbekannt, wird der Vorgang abgebrochen.
3. Server erzeugt eine frische Challenge (Zufallswert + Zeitstempel) und sendet diese an den Client.
4. Client signiert die Challenge mit dem privaten Schlüssel: $sig = sign(sk, challenge)$.
5. Client sendet sig an den Server.
6. Server prüft mit $l(s, sig) = verify(pk_s, challenge, sig)$.
7. Bei wahr ist die Authentifizierung erfolgreich.

Vorteile gegenüber dem passwortbasierten Verfahren: Der Server speichert kein Geheimnis, sondern nur den öffentlichen Schlüssel. Ein Datenleck auf Serverseite kompromittiert keine Benutzeridentitäten. Das Passwort (hier der private Schlüssel) verlässt den Client nie.

Aufgabe 3: Challenge-Response-Verfahren

a) Wieso sollte solch ein Verfahren genutzt werden?

Bei einem klassischen Passwortverfahren wird das Passwort (bzw. sein Hash) übertragen und kann mitgelesen oder wiederverwendet werden. Beim Challenge-Response-Verfahren verlässt das Passwort den Client nie. Übertragen wird nur der Hash aus Challenge und Passwort. Da die Challenge bei jeder Anmeldung wechselt, ist auch die Response jedes Mal anders. Ein Angreifer, der die Response mitliest, kann sie nicht für eine spätere Anmeldung wiederverwenden (Schutz gegen Replay).

b) Was würde geschehen, wenn die gleiche Challenge mehrfach benutzt wird?

Wird dieselbe Challenge mehrfach verwendet, ergibt sich auch stets dieselbe Response. Ein Angreifer, der die Response einmal mitgehört hat, könnte sich damit erneut anmelden (Replay-Angriff). Zusätzlich vereinfacht sich ein Offline-Angriff, da der Angreifer beliebig viele Passwort-Kandidaten gegen dieselbe Challenge/Response-Kombination testen kann. Die Sicherheit des Verfahrens wird dadurch deutlich verringert.

c) Wie kann man das verhindern?

- Die Challenge wird aus einem ausreichend großen Zufallswert (Nonce) gebildet, z. B. 128 Bit. Damit ist die Wahrscheinlichkeit einer Wiederholung praktisch null.
- Zusätzlich wird ein Zeitstempel eingebunden, der nur für ein kurzes Zeitfenster gültig ist.
- Der Server merkt sich bereits verwendete Challenges und lehnt eine wiederholte Nutzung ab.
- Jede Challenge darf nur für genau eine Anmeldung gültig sein (Einmalgebrauch).

d) Beispielhafter Ablauf

Gewählt wird die Hashfunktion SHA256. Das geteilte Passwort lautet passwort123, die Challenge ist aaa.

1. Server erzeugt Challenge $c = aaa$ und sendet sie an den Client.
2. Client bildet $r = SHA256(aaa + passwort123)$.
3. Client sendet r an den Server.
4. Server bildet selbst $r' = SHA256(aaa + passwort123)$ und vergleicht $r = r'$.

Konkrete Berechnung:

$r = SHA256(aaapasswort123)$
$r =$ 21cca91578a7ca02a253eb00d1b3526999eddcbf85cb28a0a3ec2ed2edbec3b8

Stimmen r und r' überein, ist die Anmeldung erfolgreich.

Aufgabe 4: Implementierung Challenge-Response

Aufbau

Die Klasse ChallengeResponse enthält eine innere Klasse Server und eine innere Klasse Client. Der Server verwaltet zwei Benutzer (alice und bob) mit ihren Passwörtern. Als Hashfunktion kommt SHA256 zum Einsatz, die Challenge besteht aus einer 128-Bit-Zufallszahl und einem Zeitstempel.

Ablauf im Programm

1. Client sendet Benutzernamen an den Server.
2. Server erzeugt eine frische Challenge und speichert sie zum Benutzer.
3. Client berechnet $response = SHA256(challenge \| passwort)$.
4. Server prüft die Response und markiert die Challenge als verbraucht.

Schutz gegen Replay

Der Server speichert verwendete Challenges in einem HashSet. Eine zweite Anmeldung mit derselben Challenge und Response wird abgewiesen, auch wenn die Response technisch korrekt ist. Dies verhindert einen Replay-Angriff.

Beispielausgabe

```
=== Erfolgreiche Anmeldung alice ===
Server -> Client | Challenge: d7a9a0f3...-1776284899357
Client -> Server | Response:  836e5f29...1ab9f321
Ergebnis: Login erfolgreich

=== Falsches Passwort alice ===
Ergebnis: Login abgelehnt

=== Replay-Angriff (Wiederverwendung Challenge) ===
```

```
Erste Verwendung  -> akzeptiert
Zweite Verwendung -> abgelehnt
```

Quellcode

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.*;

public class ChallengeResponse {

  private static final SecureRandom random = new SecureRandom();

  static void main(String[] args) throws NoSuchAlgorithmException {
    Server server = new Server();
    server.register("alice", "passwort123");
    server.register("bob", "geheim456");

    System.out.println("=== Erfolgreiche Anmeldung alice ===");
    login(server, "alice", "passwort123");

    System.out.println();
    System.out.println("=== Erfolgreiche Anmeldung bob ===");
    login(server, "bob", "geheim456");

    System.out.println();
    System.out.println("=== Falsches Passwort alice ===");
    login(server, "alice", "falsch");

    System.out.println();
    System.out.println("=== Replay-Angriff (Wiederverwendung Challenge) ===");
    replayAttack(server);
  }

  private static void login(Server server, String user, String password) throws
    NoSuchAlgorithmException {
    String challenge = server.requestChallenge(user);
    if (challenge == null) {
      System.out.println("Benutzer unbekannt.");
      return;
    }
    System.out.println("Server -> Client | Challenge: " + challenge);

    String response = Client.computeResponse(challenge, password);
    System.out.println("Client -> Server | Response:  " + response);

    boolean ok = server.verify(user, challenge, response);
    System.out.println("Ergebnis: " + (ok ? "Login erfolgreich" : "Login abgelehnt"));
  }

  private static void replayAttack(Server server) throws NoSuchAlgorithmException {
    String challenge = server.requestChallenge("alice");
    String response = Client.computeResponse(challenge, "passwort123");
    System.out.println("Erste Verwendung Challenge=" + challenge);
```

```java
    System.out.println("  -> " + (server.verify("alice", challenge, response) ? "akzeptiert" : "abgelehnt"));
    System.out.println("Zweite Verwendung derselben Challenge=" + challenge);
    System.out.println("  -> " + (server.verify("alice", challenge, response) ? "akzeptiert" : "abgelehnt"));
  }

  static class Server {
    private final Map<String, String> passwords = new HashMap<>();
    private final Map<String, String> openChallenges = new HashMap<>();
    private final Set<String> usedChallenges = new HashSet<>();

    public void register(String user, String password) {
      this.passwords.put(user, password);
    }

    public String requestChallenge(String user) {
      if (!this.passwords.containsKey(user)) {
        return null;
      }

      byte[] nonce = new byte[16];
      random.nextBytes(nonce);

      long timestamp = System.currentTimeMillis();
      String challenge = HexFormat.of().formatHex(nonce) + "-" + timestamp;
      this.openChallenges.put(user, challenge);
      return challenge;
    }

    public boolean verify(String user, String challenge, String response) throws NoSuchAlgorithmException {
      String expectedChallenge = this.openChallenges.get(user);
      if (expectedChallenge == null || !expectedChallenge.equals(challenge)) {
        return false;
      }

      if (this.usedChallenges.contains(challenge)) {
        return false;
      }

      String password = this.passwords.get(user);
      String expected = sha256(challenge + password);
      this.openChallenges.remove(user);

      if (expected.equals(response)) {
        this.usedChallenges.add(challenge);
        return true;
      }
      return false;
    }
  }

  static class Client {
    public static String computeResponse(String challenge, String password) throws NoSuchAlgorithmException {
      return sha256(challenge + password);
    }
  }
```

```java

  private static String sha256(String input) throws NoSuchAlgorithmException {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] hash = md.digest(input.getBytes(StandardCharsets.UTF_8));
    return HexFormat.of().formatHex(hash);
  }
}
```

Code Listing 1: ChallengeResponse.java